Parallel BFS and DFS using OpenMP

1. Aim of the Practical

To design and implement:

- **Parallel Breadth First Search (BFS)**
- **Parallel Depth First Search (DFS)**

using:

- **OpenMP**
- Graph data structure

The program performs graph traversal in parallel using multiple threads.

---

2. What is Graph Traversal?

Graph traversal means visiting all nodes (vertices) of a graph.

Two important traversal algorithms are:

Algorithm	Traversal Style
BFS	Level by level
DFS	Depth wise

---

3. What is BFS?

BFS = **Breadth First Search**

It visits:

- First all neighbors
- Then neighbors of neighbors

Uses:

- Queue
- FIFO principle

BFS Example

Graph:

```
  0
 / \
```

```
  1   2
 /     \
3       4
 \     /
   5
```

BFS from 0:

0 → 1 → 2 → 3 → 4 → 5

---

4. What is DFS?

DFS = **Depth First Search**

It visits nodes deeply before backtracking.

Uses:

- Stack
- LIFO principle

DFS Example

DFS from 0:

0 → 2 → 4 → 5 → 3 → 1

(Traversal may vary)

---

5. What is Parallel Computing?

Parallel computing means:

- Multiple tasks execute simultaneously
- Uses multiple CPU cores

Benefits:

- Faster execution
- Better performance

---

6. What is OpenMP?

OpenMP = **Open Multi-Processing**

It is an API used for parallel programming in C/C++.

It creates multiple threads automatically.

OpenMP Syntax

#pragma omp parallel for

This converts loop into parallel loop.

---

7. Header Files Used

#include <iostream>

#include <queue>

#include <stack>

#include <omp.h>

#include <vector>

---

8. Purpose of Header Files

Header	Purpose
iostream	Input/output
queue	BFS queue
stack	DFS stack
omp.h	OpenMP functions
vector	Dynamic arrays

---

9. Creating Graph Class

class Graph{

Class contains:

- Number of vertices
- Adjacency list
- BFS and DFS functions

---

10. Graph Representation

vector<vector<int>> adj;

This is:

Adjacency List Representation

Example

If edge exists:

0 — 1

0 — 2

Adjacency list:

adj[0] = {1,2}

adj[1] = {0}

adj[2] = {0}

---

11. Constructor

```
Graph(int v){
   this->v = v;
   adj.resize(v);
}
```

Purpose

- Stores number of vertices
- Creates adjacency list

---

12. Adding Edge

```
void addEdge(int a, int b){
   adj[a].push_back(b);
   adj[b].push_back(a);
}
```

This creates:

Undirected Graph

Because:

- a connects to b
- b connects to a

---

13. Parallel BFS Function

void parallelBfs(int start)

Performs Breadth First Search in parallel.

---

14. Queue Creation

queue<int> q;

Queue is used because BFS follows:

FIFO (First In First Out)

---

15. Visited Array

vector<bool> visited(v,false);

Purpose:

- Prevent revisiting nodes
- Avoid infinite loops

Initially all nodes:

false

---

16. Starting Node

visited[start] = true;

q.push(start);

Start node:

- Marked visited
- Inserted into queue

---

17. BFS Loop

while(!q.empty())

Loop runs until queue becomes empty.

---

18. Removing Front Node

int node = q.front();

q.pop();

Gets current node from queue.

---

19. Printing Node

cout << node <<"->";

Displays traversal order.

---

20. Parallel Region in BFS

#pragma omp parallel for

MOST IMPORTANT PART

This parallelizes:

for(int i=0;i<adj[node].size();i++)

Meaning:

- Multiple neighbors processed simultaneously.

---

21. Neighbor Traversal

int neighbours = adj[node][i];

Gets adjacent node.

---

22. Checking Visited Node

if(!visited[neighbours])

Ensures:

- Node processed only once

---

23. Critical Section

#pragma omp critical

VERY IMPORTANT CONCEPT

Why Critical Section Needed?

Multiple threads may:

- Access queue simultaneously

- Modify visited array simultaneously

This causes:

Race Condition

Race Condition

Occurs when:

- Multiple threads modify same data together

Can produce incorrect results.

Critical Section Solves This

Only one thread enters critical block at a time.

```
#pragma omp critical
{
   visited[neighbours] = true;
   q.push(neighbours);
}
```

Ensures thread safety.

---

24. Parallel DFS Function

void parallelDfs(int start)

Performs DFS traversal using OpenMP.

---

25. Stack in DFS

stack<int> st;

DFS uses:

LIFO (Last In First Out)

---

26. Push Starting Node

st.push(start);

Starting vertex inserted into stack.

---

27. DFS Loop

while(!st.empty())

Runs until stack becomes empty.

---

28. Removing Top Element

int node = st.top();

st.pop();

Gets current node from stack.

---

29. Parallel DFS Traversal

#pragma omp parallel for

Processes neighbors in parallel.

---

30. DFS Critical Section

#pragma omp critical

Protects:

- Stack access
- Visited array updates

---

31. Main Function

int main()

Program execution starts here.

---

32. Creating Graph

Graph g(6);

Creates graph with:

6 vertices

Vertices:

0 to 5

---

33. Adding Edges

```
g.addEdge(0,1);
g.addEdge(0,2);
g.addEdge(1,3);
g.addEdge(2,4);
g.addEdge(3,5);
g.addEdge(4,5);
```

Creates graph structure.

---

34. Calling BFS

```
g.parallelBfs(0);
```

Starts BFS from node 0.

---

35. Calling DFS

```
g.parallelDfs(0);
```

Starts DFS from node 0.

---

36. Expected Output

BFS Output

```
parallel Bfs using openmp
0->1->2->3->4->5->
```

DFS Output

```
parallel dfs using openmp
0->2->4->5->3->1->
```

(DFS order may vary)

---

37. Why DFS Order Changes?

Because:

- Parallel threads execute differently
- Scheduling may vary

Hence DFS output is not always fixed.

---

38. Time Complexity

BFS

[
O(V + E)
]

DFS

[
O(V + E)
]

Where:

- (V) = vertices
- (E) = edges

---

39. Advantages of Parallel BFS/DFS

Advantage	Description
Faster execution	Multiple threads work together
Efficient traversal	Large graphs processed quickly
Better CPU utilization	Multi-core usage

---

40. Limitations

Problem	Reason
Synchronization overhead	Critical sections slow threads
Race conditions	Shared memory conflicts
Non-deterministic DFS	Thread scheduling changes output

---

41. Important OpenMP Concepts Used

Concept	Purpose
parallel for	Parallel loop execution

Concept	Purpose
critical	Prevent race conditions
threads	Multiple parallel workers

---

42. Compilation Command

To compile OpenMP code:

Linux / GCC

g++ file.cpp -fopenmp

Run:

./a.out

---

43. Important Viva Questions

Q1. What is BFS?

BFS traverses graph level by level using queue.

Q2. What is DFS?

DFS traverses graph deeply using stack.

Q3. Why use OpenMP?

To achieve parallel execution using multiple CPU cores.

Q4. Why use critical section?

To avoid race conditions while accessing shared data.

Q5. What is race condition?

When multiple threads modify shared data simultaneously causing incorrect results.

Q6. Why adjacency list is used?

Efficient memory usage for sparse graphs.

Q7. Difference between BFS and DFS?

BFS	DFS
Uses Queue	Uses Stack
Level traversal	Depth traversal
Finds shortest path	May not find shortest path

---

44. Final Conclusion

In this practical:

- Graph traversal algorithms BFS and DFS were implemented
- OpenMP enabled parallel execution
- Parallel loops improved performance
- Critical sections ensured thread safety

The program demonstrates how parallel computing can accelerate graph traversal algorithms using multi-threading.